

Callback

Übungen

Youtube

Implementiere die Methoden in User mit Hilfe von Callbacks, sodass die kommentierte Ausgabe ausgegeben wird.

```
class Youtube {  
  
    /**  
     * Implement the methods in {@link User} by using Callbacks so that the following output is printed.  
     */  
    public static void main(String[] args) {  
        var elona = new User("Elona");  
        var anna = new User("Anna");  
        var jeff = new User("Jeff");  
        var sara = new User("Sara");  
  
        elona.subscribe(anna);  
        jeff.subscribe(anna);  
        elona.subscribe(jeff);  
        jeff.subscribe(elon);  
        sara.subscribe(elon);  
  
        elona.upload(new Video("If You Feel Burnt Out Do This"));  
        jeff.upload(new Video("The Right Way To Build REST APIs"));  
        anna.upload(new Video("Try on haul 2024"));  
  
        /**  
         Sara has been notified of a new video 'If You Feel Burnt Out Do This' by Elona  
         Jeff has been notified of a new video 'If You Feel Burnt Out Do This' by Elona  
         Elona has been notified of a new video 'The Right Way To Build REST APIs' by Jeff  
         Elona has been notified of a new video 'Try on haul 2024' by Anna  
         Jeff has been notified of a new video 'Try on haul 2024' by Anna  
         */  
    }  
}
```

```
class User {  
  
    private final String name;  
    private final Collection<Video> videos = new ArrayList<>();  
  
    User(String name) {  
        this.name = name;  
    }  
  
    void upload(Video video) {  
        videos.add(video);  
        // TODO  
    }  
  
    private void notifySubscriberOfNewVideo(User uploader, Video video) {  
        System.out.printf("%s has been notified of a new video '%s' by %s%n", name, video, uploader);  
    }  
  
    void subscribe(User subscriber) {  
        // TODO  
    }  
  
    Collection<Video> allUploadedVideos() {  
        return Collections.unmodifiableCollection(videos);  
    }  
  
    @Override  
    public String toString() {  
        return name;  
    }  
}
```

```
record Video(String title) {  
  
    @Override  
    public String toString() {  
        return title;  
    }  
}
```

Call of Shooty: Code

Gegeben ist folgender Code.
Schreibe einen Unit Test mit 100% Testabdeckung
für die Klasse **GameEngine**.

```
@FunctionalInterface
interface PlayerEliminated {

    void playerEliminated(Player eliminator, Player eliminatee);
}
```

```
class Player {

    private final String name;
    private int score;

    Player(String name) {
        this.name = name;
    }

    String name() {
        return name;
    }

    int score() {
        return score;
    }

    void incrementScore() {
        score++;
    }

    @Override
    public String toString() {
        return "Player{name=\"" + name + "\" + ", score=" + score + "}";
    }
}
```

```
import java.util.*;

class GameEngine {

    private static final int PLAYERS_TO_CREATE = 10;
    private static final Random RANDOM = new Random();

    private final List<Player> players = new ArrayList<>();
    private final Collection<PlayerEliminated> playerEliminatedSubscribers = new ArrayList<>();

    GameEngine() { createPlayers(); }

    private void createPlayers() {
        for (int i = 0; i < PLAYERS_TO_CREATE; i++) {
            String defaultName = "Player" + i;
            players.add(new Player(defaultName));
        }
    }

    void subscribePlayerEliminated(PlayerEliminated subscriber) {
        playerEliminatedSubscribers.add(subscriber);
    }

    void notifyPlayerEliminated(Player eliminator, Player eliminatee) {
        playerEliminatedSubscribers.forEach(subscriber -> subscriber.playerEliminated(eliminator, eliminatee));
    }

    void run() {
        Player eliminator = getRandomPlayer();
        Player eliminatee = getRandomPlayer();
        if (eliminator != eliminatee) {
            eliminator.incrementScore();
        }
        notifyPlayerEliminated(eliminator, eliminatee); // eliminator has just shot eliminatee
    }

    private Player getRandomPlayer() { return players.get(randomPlayerIndex()); }

    private int randomPlayerIndex() { return RANDOM.nextInt(0, players.size()); }
}
```



Student Modified Callback



Gegeben ist die Klasse **Student** inklusive **Unit Test** (siehe nächste Folie).

Ergänze die Klasse Student, sodass man **beliebig viele Callbacks** hinzufügen kann, die aufgerufen werden, sobald sich irgendeines der Attribute in Student geändert hat (subscribeOnModified).

Die hinzugefügten Callbacks werden **evaluiert**, **sobald** sich eines der **Attribute** in Student **ändert** – also sobald einer der Setter aufgerufen wird. Der Callback soll den **geänderten Student** als Parameter erhalten.

Teste deine **Callbacks**, indem du den **Unit Test erweiterst**.

// Beispiel:

```
public static void main(String[] args) {  
    var student = new Student(1, "Elon");  
    student.subscribeOnModified(modifiedStudent -> System.out.println(modifiedStudent));  
    System.out.println(student);  
    student.setId(2); // evaluate callback  
    student.setName("Elon Musk"); // evaluate callback  
}
```

Student Change: Code

```
class Student {  
  
    private int id;  
    private String name;  
  
    Student(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    int id() {  
        return id;  
    }  
  
    void setId(int id) {  
        this.id = id;  
    }  
  
    String name() {  
        return name;  
    }  
  
    void setName(String name) {  
        this.name = name;  
    }  
  
    @Override  
    public String toString() {  
        return "Student{id=" + id + ", name=" + name + "\n" + '}';  
    }  
}
```

```
class StudentTest {  
  
    private Student student;  
  
    @BeforeEach  
    void setUp() {  
        student = new Student(1, "Elon");  
    }  
  
    @Test  
    void getSetId() {  
        student.setId(2);  
  
        assertEquals(2, student.id());  
    }  
  
    @Test  
    void getSetName() {  
        student.setName("Elon Musk");  
  
        assertEquals("Elon Musk", student.name());  
    }  
  
    @Test  
    void toStringWithIdAndName() {  
        var actual = student.toString();  
  
        assertEquals("Student{id=1, name='Elon'}", actual);  
    }  
}
```

Big Brother



Gegeben sind folgende Klassen. Vervollständige den Code an den mit // TODO kommentierten Stellen, sodass der unveränderte Unit Test erfolgreich besteht.

```
import java.util.LinkedList;
import java.util.List;

class BigBrother {

    private final List<String> log = new LinkedList<>();

    BigBrother add(Person person) {
        // TODO
    }

    boolean logContains(String line) {
        return log.contains(line);
    }
}
```

```
@FunctionalInterface
interface NameChangedCallback {

    void namedChanged(String oldName, String newName);
}
```

```
import java.util.LinkedList;
import java.util.List;

class Person {

    // TODO

    private String name;

    Person(String name) {
        this.name = name;
    }

    String name() {
        return name;
    }

    void setName(String name) {
        // TODO
    }

    void subscribeNameChanged(NameChangedCallback subscriber) {
        // TODO
    }
}
```

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class BigBrotherTest {

    @Test
    void subscribeAndNotifyOnNameChanged() {
        Person elon = new Person("Elon");
        Person sara = new Person("Sara");

        BigBrother bigBrother = new BigBrother()
            .add(elon)
            .add(sara);

        elon.setName("Musk");
        assertTrue(bigBrother.logContains("Elon changed name to Musk"));

        sara.setName("Lara");
        assertTrue(bigBrother.logContains("Sara changed name to Lara"));
    }

    @Test
    void notNotifyIfNotSubscribedOnNameChanged() {
        Person elon = new Person("Elon");

        BigBrother bigBrother = new BigBrother();

        elon.setName("Musk");
        assertFalse(bigBrother.logContains("Elon changed name to Musk"));
    }
}
```